

Project Report
On
Hybrid Chatbot and Aspect-Based Sentiment Analysis Architecture

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computers Science and Engineering

2025-2026

By

Anwasha Singh

23FE10CSE00299

Oshika Sharma

23FE10CSE00145



**MANIPAL UNIVERSITY
JAIPUR**

Signature of Supervisor

**Department of Computer Science and Engineering
School of Computer Science and Engineering
Manipal University Jaipur
Jaipur, Rajasthan - 303007**

1. INTRODUCTION OF PROBLEM STATEMENT

The boom of unstructured data (especially textual) in the digital world, has led to increased importance of developing advanced Natural Language Processing systems. It is even more important in e-commerce sectors, where organizations are continuously receiving user reviews and feedback. Natural Language Processing is a sub-field of Artificial Intelligence that deals with the interactions between computers and human language. It majorly focuses on programming computers to process and analyse large amounts of natural language data. Users and consumers generate humongous amounts of feedback, product reviews, and conversational queries daily. Chatbots have been used to handle direct user queries and sentiment analysis engines are used to predict customer satisfaction through review analysis.

Traditional chatbots depend on a rule-based knowledgebase that completely fails to read the context of the user's sentiment. Traditional sentiment analysis systems often use document-level categorisation, assigning a single sentiment (positive, negative, or neutral) to an entire document or text chunk. This evaluation fundamentally misinterprets the complexity of human feedback.

This project works on a unified Hybrid Chatbot integrated with an Aspect-Based Sentiment Analysis framework. This system is capable of deciding whether a query from a user is conversational or review based and then responds accordingly. By using Natural Language Processing libraries, such as spacy, scikit-learn, langgraph, and langchain, the system processes natural language inputs to extract user intent and extract aspect-level sentiments, which somewhat does justice to complex intents and sentiments in human language.

Using this pipeline, the system extracts linguistic features and then routes data to either conversational intent classifier or the Aspect-Based Sentiment Analysis module.

1.1 Architectural Framework and the Routing Mechanism

The efficiency of this hybrid system lies in the “Routing Architecture”. In conventional chatbots, the user query is submitted via a chat interface, which is then processed by a dialogue engine. If the query contains a product review, then it is added to the database and is batch processed by a sentiment analysis engine. This long process increases computational overhead and latency.

This system eliminates redundancy. Input data is cleaned and processed in a staging area. After all the transformations, a router (algorithmic module) determines the primary nature of the text and directs the computational flow as required.

2. CONTRIBUTION (discussed in detail later)

2.1. Custom Aspect-Based Sentiment Logic (Anwesha Singh)

Implemented

- Window-based context analysis
- Negation handling
- Contrast handling using "but"
- Overall Sentiment
- Aspect-Based Sentiment

2.2. Chatbot Intent Engine (Oshika Sharma)

- Hybrid preprocessing using NLTK and Spacy
- Automated Data Augmentation
- TF-IDF Vectorization
- Model Benchmarking: Logistic Regression, Random Forest, Naive Bayes, KNN, Gradient Boosting, and Voting Ensembles.

Final model selected: Linear SVM (showed best performance).

2.3. Hybrid Unified Architecture

A single pipeline including:

- Preprocessing
- TF-IDF feature extraction
- Smart routing
- Chatbot + Sentiment + ABSA.

3. DATASET CHARACTERISTICS AND EXPLORATORY DATA ANALYSIS

The two module architecture of this hybrid model needs two completely separate and different datasets. The chatbot requires an intent classification corpus for its training. Whereas, the Aspect-Based Sentiment Analysis module needs a product review dataset for training and testing.

3.1 The Chatbot Intent Classification Corpus

The conversation oriented chatbot node was trained on an intent dataset that was custom made specifically for this project. In AI and Natural Language Processing, intent is understood as the goal behind the user's interaction. For appropriate generation of response, understanding and identifying this intent is a crucial step.

The corpus that is used for training intent classifier is custom-made, and is compact and balanced. The corpus has 117 training sentences; these sentences are also called patterns when dealing with chatbot training. These patterns are mapped with 8 distinct intent labels.

The labels are given in the table below:

Label	Pattern Count	Percentage of Total Corpus	Description
ai_application	21	17.94%	Queries relating to the applications of Artificial Intelligence.
tfidf_concept	20	17.09%	Technical queries regarding TF-IDF mathematics.
nlp_core	18	15.38%	Queries regarding the fundamental concepts and algorithms of Natural Language Processing.
greeting	15	12.82%	Standard conversational openers (e.g., "Hello," "Hi," "Good morning").
goodbye	13	11.11%	Standard conversational closers (e.g., "Bye," "See you later," "Farewell").
Other Intents	30	25.64%	
Total	117	100.00%	The complete training corpus for the conversational model.

Observing the numbers above, these intent patterns are oriented towards academic and technical aspects of AI and NLP, they cover major technologies this very system is built on. Typical

conversation starters and conclusers like greetings and goodbyes account about 24% of the corpus. This ensures that the chatbot has a polite and natural tone during conversation.

3.2 The E-commerce Product Review Corpus

For the Aspect Based Analysis, a dataset with customer reviews (flipkart_product.csv) was used. This dataset has messy, unstructured data, it also has contradictory natural language reviews that conventional models fail to process accurately.

1. **ProductName:** Contains the full name of the product, it includes technical details, and capacity of the products.
2. **Price:** Lists the cost of the product.
3. **Rate:** A score between 1 to 5, representing user satisfaction.
4. **Review:** A sentiment headline summarising the user's feeling (e.g., "Super!", "Useless product", "Mind-blowing purchase").
5. **Summary:** The most important column for Aspect-Based Sentiment Analysis. This contains the detailed description where users provide feedback regarding features of the product.

Positive reviews use phrases such as "excellent air flow," "smart cool breeze producer," and "massive cooling capability". Negative reviews highlight the limitations.

These feedbacks show unpredictable and multi factor behaviour, which is why document-level sentiment analysis fails to capture emotions of such product reviews. For example, a review saying. “ The quality is good, but battery life of the product is not nice”, this review contains contradictory reviews about different aspects of the product and also uses negation; A traditional algorithm would classify the whole sentence as “neutral” which doesn’t justify the complexity of this review. To extract the sentiment about these specific aspects need a custom, special rule-based Aspect-Based Sentiment Analysis logic included in the system.

4. LINGUISTIC PREPROCESSING AND FEATURE ENGINEERING

As machine learning algorithms cannot understand English words, therefore natural language text is converted to numerical vectors using normalisation and mathematical transformations to standardize the text and perform prediction.

4.1 Text Normalisation

During the preprocessing sequence of operations are executed to reduce the dimensionality of the data and remove any linguistic noise. The steps used are:

1. **Lowercasing:** All textual input is uniformly converted to lowercase.
 - Standardized terms like "COOLER" and "cooler" into a single token.
 - "What is NLP?" and "what is nlp" like sentences were mapped to one vector
2. **Punctuation Removal:** Non-alphanumeric characters, such as commas, periods, and exclamation marks, removed from the text.
 - Noise like "!!!" or "..." is removed from the review
 - "why is NLP?" and "why is nlp" like queries are mapped in one vector
3. **Stopword Removal:** Highly frequent, semantically useless words such as "the", "is", "in", "and", and "at" are removed.
 - Helped to focus on sentiment-bearing words like "good", "bad", "waste", etc.
 - Terms like "tokenization" or "tfidf" received higher mathematical weight
4. **Lemmatization:** Focuses on deep morphological analysis to resolve words back to their dictionary root.
 - Grouped different mentions for the same product feature like "drain" and "draining".
 - Used Spacy in built feature which was effective for "greetings" and "NLP_core"

```
stop_words = set(stopwords.words('english'))

lemmatizer = WordNetLemmatizer()

def preprocess(text):
    if pd.isna(text):
        return ""
    text = str(text).lower() # Convert to lowercase
    text = re.sub(r'^a-z\s', '', text) #Removing special characters
    words = text.split()
    words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words and w != 'nan'] #Stopword removal, lemmatization
    return " ".join(words)

df['clean_text'] = df['text'].apply(preprocess)
```

```
Data Preparation Complete!
Total Unique Patterns: 117
Labels counts: ai_application    21
tfidf_concept      20
nlp_core           18
greeting           15
goodbye            13
tokenization       11
lemmatization      10
stemming           9
Name: count, dtype: int64
```

4.2 Vectorisation (TF-IDF)

After the normalisation, the text is mathematically quantified. The system uses Term Frequency-Inverse Document Frequency (TF-IDF) vectorisation algorithm.

TF-IDF corrects this imbalance by combining two distinct statistical metrics to calculate a weight for every single word:

1. **Term Frequency (TF):** Measured how frequently a word appeared

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. **Inverse Document Frequency (IDF):** Words that appear in almost every document (like "good" or "product") are given a low IDF score. Rare words that appear frequently only in a few specific documents (such as "humid", "cross-ventilation", or "soundless") receive a higher IDF weight.

$$IDF(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

Multiplied these two metrics together to produce a feature matrix where the highest numerical values are assigned to words that are uniquely defining characteristics of a specific intent and sentiment.

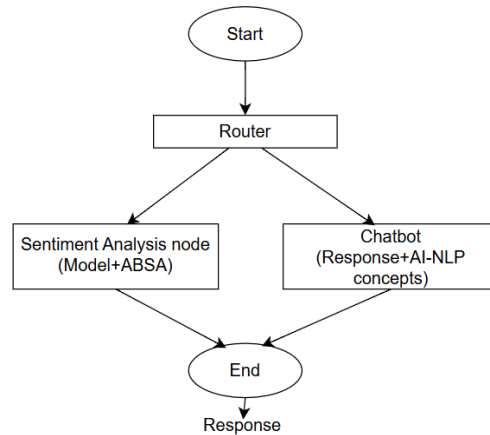
$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

▶ # Analysing both words and character chunks (3-5 chars)

```
vectorizer = TfidfVectorizer(  
    ngram_range=(1, 2), # 1-word and 2-words  
    max_features=1500,  
    sublinear_tf=True  
)  
X = vectorizer.fit_transform(training_sentences)
```


5. THE ALGORITHMIC LOGIC OF THE ROUTING MECHANISM

The router operates as the orchestrator of this system. It uses a keyword-based heuristic to classify the domain of the text.



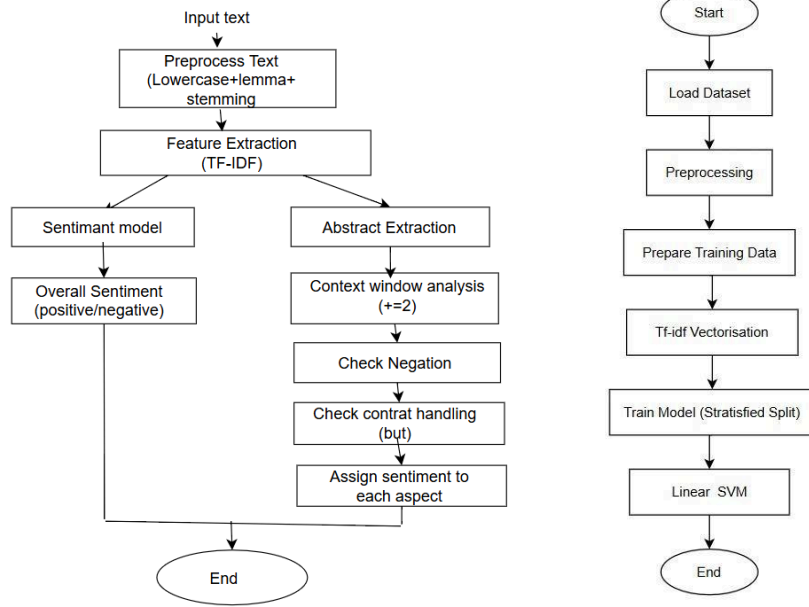
The routing logic executes a sequence of operations:

First, the user's raw natural query is input into the system and linguistically cleaned by the preprocessing node. This removes noise and reduces it to its fundamental roots (morphemes). Then the text is transformed into a numerical matrix using Term Frequency-Inverse Document Frequency vectoriser. This ensures that both the modules, that is chatbot and the sentiment analysis modules operate on the same mathematical representation of the user's input.

Second, an initial intent prediction is done. A Support Vector classifier evaluates the vectorised text to predict the baseline conversational intent of the user. This gives the system an initial understanding of whether the user is asking a question, greeting the bot, or seeking technical definitions.

Third, the most important step of the routing mechanism is keyword-based bifurcation. The router parses the preprocessed text for specific morphemes of product related feature keywords. The router searches for the terms such as "battery", "screen", "camera", "display", etc.

Finally, if the user's text contains any of the predefined target keywords, then the system classifies it as a conversational or review-based query. The text is then routed to either conversational chatbot or the Aspect-Based Sentiment Analysis module. If the text does not have the specific keywords, the system routes the input towards general conversation related queries and standard intent-based chatbot generates an appropriate conversational response.



This routing mechanism ensures that computational resources are allocated with maximum efficiency. The integration of advanced workflow orchestration libraries such as langchain and langgraph facilitates this workflow.

6. HYPERPARAMETER TUNING

Hyperparameter tunings to further optimise performance:

- **ngram_range = (1, 2):** Configured to capture both unigrams and bigrams. A pure unigram model would parse the phrase "not good" into two completely separate, unrelated features, that is "not" and "good". This might lead to associating the word "good" with positive sentiment. By including bigrams, the model captures "not good" as a single feature, this allows machine learning algorithms to correctly identify the negative sentiment.
- **max_features:** By ensuring that only the most significant N-grams are retained, the system avoids the curse of dimensionality and mitigates the risk of overfitting the model on the highly limited 117-pattern intent dataset.

7. ALGORITHMIC EVALUATION OF INTENT CLASSIFICATION

The intent classification part of the chatbot is responsible for determining the user's goal, ensuring that queries regarding general questions are not mixed with product reviews. Sparse TF-IDF matrix and small sample size leads to the need of carefully selecting the correct machine learning algorithm.

To identify the best suited model, multiple different classical machine learning algorithms were implemented (Linear Support Vector Machine (SVM), Voting Ensemble, Logistic Regression, Random Forest, K-Nearest Neighbours (KNN), and Naive Bayes). Neural networks and Large Language Models were not chosen for this task because, for such a small dataset they might overfit and they have high latency.

7.1 Results of Model Comparison

The models were trained on the TF-IDF feature vectors of the intent corpus, and their classification accuracy was evaluated .

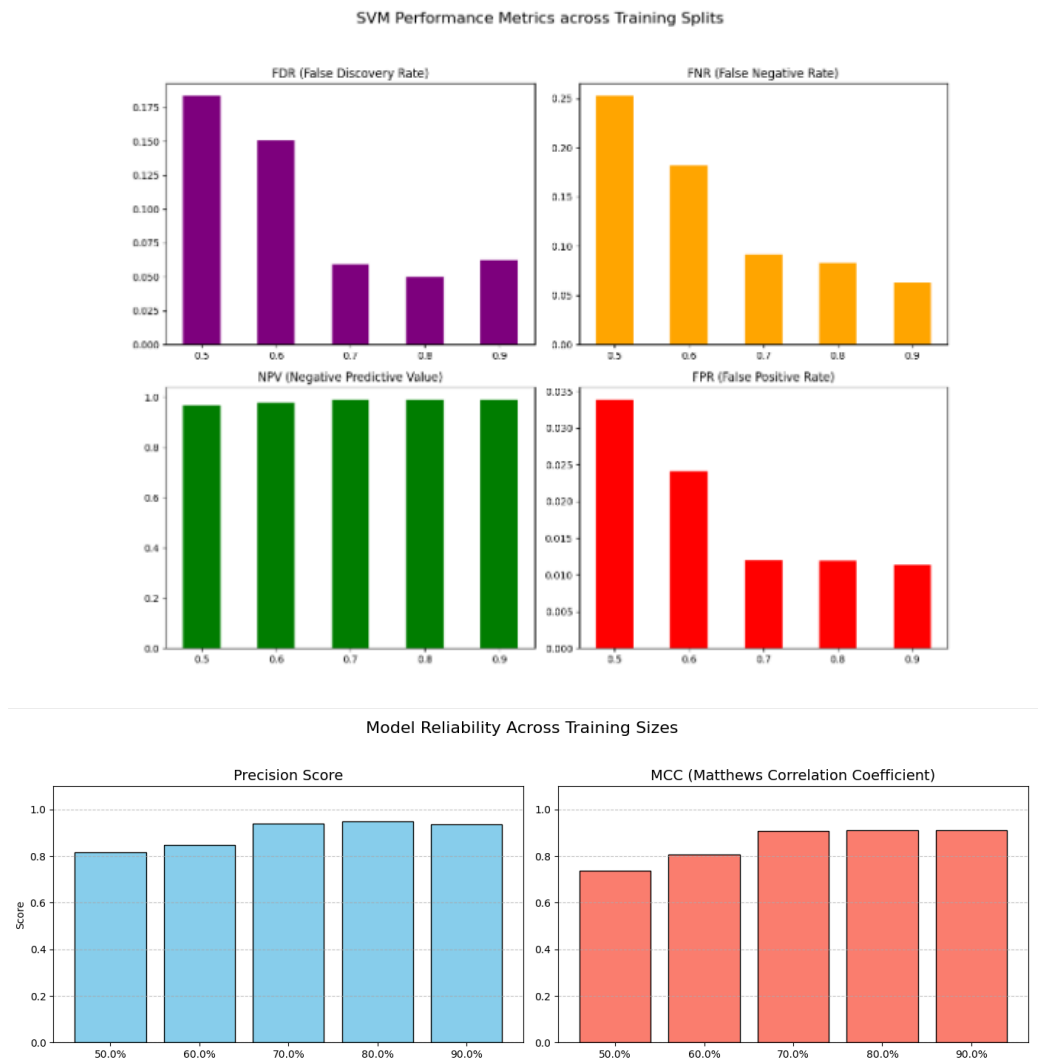
Machine Learning Algorithm	Overall Accuracy
Linear Support Vector Machine (SVM)	0.917
Voting Ensemble	0.917
Logistic Regression	0.833
Random Forest	0.750
K-Nearest Neighbours (KNN)	0.708
Naive Bayes	0.542

Because the Linear SVM and the Voting Ensemble achieved best accuracies (91.7%), the Linear SVM was selected as the final model for router.

- **Accuracy:** 0.9167
- **Precision:** 0.9405
- **Recall:** 0.9083
- **F1 Score:** 0.9091
- **Specificity:** 0.9880
- **Matthews Correlation Coefficient (MCC):** 0.9076

Accuracy: 0.9167
 Precision: 0.9405
 Recall: 0.9083
 Specificity: 0.9880
 F1 Score: 0.9091
 FPR: 0.0120
 FNR: 0.0917
 NPV: 0.9887
 FDR: 0.0595
 MCC: 0.9076

Graphical analyses of the model indicated a Negative Prediction Value (NPV) approaching 1.0 and False Positive Rate (FPR) ranging between 0.01 and 0.03.



7.2 Linear SVM versus Naive Bayes

The gap in accuracies of Linear SVM and Naive-Bayes accurately shows the limitations of working with sparse textual data. Historically, Naive-Bayes has been considered the baseline standard for text classification. It is computationally efficient, shows faster training time, and is based on a simple probabilistic algorithm, that is Bayes' Theorem.

```

Accuracy: 0.9167
Precision: 0.9405
Recall: 0.9083
Specificity: 0.9880
F1 Score: 0.9091
FPR: 0.0120
FNR: 0.0917
NPV: 0.9887
FDR: 0.0595
MCC: 0.9076

```

Linear SVM metrics

```

Accuracy: 0.9
Precision: 0.8
Recall: 1.0
Specificity: 0.8333333333333334
F1 Score: 0.8888888888888888
FPR: 0.16666666666666666
FNR: 0.0
NPV: 1.0
FDR: 0.2
MCC: 0.816496580927726

```

Naïve-Bayes metrics

The major limitation of Naive-Bayes is that this algorithm operates on some assumptions, like all the features (all TF-IDF n-grams) are completely independent of one another for the given class label. But for natural language, this assumption is completely wrong. In natural language, words are very correlated. The chatbot's intent corpus relies heavily on specific technical labels (eg. `ai_application`, `tfidf_concept`), hence the features extracted by TF-IDF vectoriser are interdependent. Naive-Bayes cannot capture these interdependencies, which leads to strict degradation in accuracy.

On the other hand, Support Vector Machines work on geometric paradigms. A SVM maps the TF-IDF feature vectors to a multi-dimensional space. SVM's speciality is that it works well on sparse data, which works well with feature matrices made by TF-IDF where most values are observed to be 0. The reason being, optimal hyperplane is determined by a small subset of datapoints located closest to the support vectors.

7.3 Aspect-Based Sentiment Analysis (ABSA)

The Support Vector Machine effectively handles conversational intents and the analytical part is done by the custom built Aspect-Based Sentiment Analysis (ABSA) model. The standard document-level sentiment allocation fails on rich multi-factor sentences, which are very common for e-commerce product reviews.

The system uses a carefully optimized, rule-based custom ABSA function. Even though LLMs are powerful, a rule based approach always offers a better execution speed, transparency in decision making, and no hallucination, and it is feasible for such a small dataset.

This custom ABSA function uses three processes to dissect text:

7.3.1. Window-Based Context Extraction

When a target aspect is identified in a sentence, the adjective modifying that aspect is usually located in close proximity to that target aspect. It is rarely located at the far end of the paragraph or sentence. The logic used by ABSA applies window based heuristic, which is localized.

The algorithm focuses on the context window consisting exactly 2 words before and 2 words after the target aspect. This constraint uses the well known principle of semantic proximity. By eliminating text outside the window, the algorithm ends up focusing on the adjectives and adverbs that actually modify the aspect.

7.3.2. Negation Handling Logic

Negation has always been a complex challenge in natural Language Processing, because it has the capabilities of completely inverting the intent of the surrounding text. Algorithms like Bag-of Words will never be able to capture such patterns.

The custom ABSA function keeps this in consideration and uses a rule-based negation detector. Once the context windows are extracted, the algorithm parses the context window for very specific negation tokens like: ‘not’, ‘don’t’, and ‘didn’t’. This rule is hardcoded into the function.

Once such token is detected in the context window in the proximity of sentiment carrying adjective, the function mathematically flips the sentiment multiplier. This ensures negation affected phrases are evaluated accurately, and false positive sentiment tagging is avoided.

7.3.3. Contrast Dependency Parsing

E-commerce reviews often have coordinating conjunctions to switch to a contradicting sentiment in a single compound sentence. To deal with this, the algorithm scans the text for the presence of the token “but”. The algorithm considers the fact that the clause after the contrast token must be evaluated independently.

8. CONCLUSION

This project presents a methodology to make a hybrid conversational artificial intelligence with opinion mining. Using the smart router alongside the TF-IDF feature extraction process, the system reduces the redundancy and latency.

The analysis shows that for high-dimensional, sparse textual features (eg. those generated by TF-IDF vectorization), algorithms like Linear SVM perform better than models like Naive-Bayes. The SVM achieved an accuracy of 91.67%, F1 score of 0.9091, and specificity of 0.9880. This shows the reliability of the routing function. Challenge and major limitation faced in this project is the sparsity of the intent corpus itself.

The custom rule based ABSA provides an alternative with low latency, instead of deploying a heavy neural network for every customer request. Using the 2 word bidirectional context window and hardcoding grammatical heuristics for negation and contrast handling, the ABSA function accurately bifurcates complex and contradictory customer reviews.

Github repository: <https://github.com/Anwasha-code/natural-language-processing-concepts>

Colab-notebook(execution):

<https://colab.research.google.com/drive/1Lav6CymNtevYlbdPbgs5KPjx8tZaUaVR>